

Autonomous Forest Data Collection

Pedro Ferreira, nmec: 98620

1. Introduction	
2. System Architecture	
3. Implementation	
3.1. Entities	
3.1.1. Base Station	
3.1.2. Drone	
3.1.3. Sensor	
3.2. Proximity Manager	
3.3. Cooperative Awareness Messages (CAM)	
3.4. Decentralized Environmental Notification Messages (DENM)	
3.5. Coverage Algorithms	
3.5.1. Boustrophedon	
3.5.2. Greedy Nearest	
3.5.3. Frontier Exploration	
3.6. Simulation	
4. Results	
5. Conclusion	
5.1. Future Work	

1. Introduction

Forest monitoring is a critical task for environmental preservation, early fire detection, and wildlife tracking. However, performing continuous, large-scale coverage of remote terrain is a challenge, as human inspection is typically costly, slow, and potentially dangerous. To address these issues, this project introduces a simulated environment for autonomous forest data collection, where a fleet of drones sweeps a geographical area to detect static ground sensors, collect environmental readings, and relay this data to a base station.

The primary objectives of this project are:

1. To establish a decentralized coordination mechanism for multiple drones without reliance on centralized path planning.
2. To use Vehicle-to-Everything (V2X) communication standards, specifically Cooperative Awareness Messages (CAM) and Decentralized Environmental Notification Messages (DENMs).
3. To implement, evaluate, and compare three distinct coverage strategies (Boustrophedon, Greedy Nearest, and Frontier Exploration) under identical simulation conditions.
4. To provide an interactive dashboard for real-time observation of the system state, network connectivity, coverage progress, and data deliveries.

The complete implementation of the system described in this report is publicly available at: <https://github.com/PedroDSFerreira/rsa-project>

2. System Architecture

The system is designed as a distributed, microservices-oriented architecture, deployed as a set of containerized Docker services.

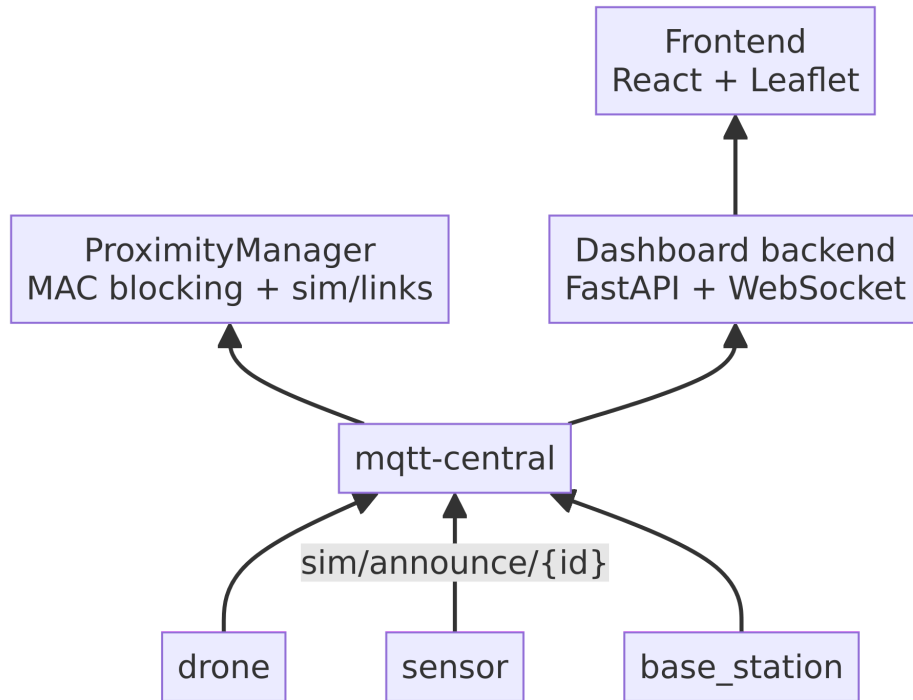


Figure 1: System architecture overview.

The architecture comprises the following key components:

- **MQTT Broker (Mosquitto):** Acts as the underlying transport layer for the V2X simulation topics, providing the control and data plane for message exchange between all entities.
- **Base Station:** Serves as the mission coordinator. It initiates missions, assigns initial state requirements if dictated by the algorithm, and aggregates sensor data deliveries upon mission completion.
- **Drone Replicas:** Autonomous agents responsible for executing the coverage algorithm. They broadcast their presence, coordinate via state updates, navigate the grid, and request sensor data.
- **Sensor Replicas:** Static ground entities that announce their position and serve environmental data upon receiving a proximity-based request.
- **Proximity Manager:** A service that simulates a realistic radio range model. By continually computing the distances between all entities, it dynamically determines and publishes which communication links are active, effectively acting as a spatial filter for message dissemination.
- **Dashboard Backend & Frontend:** A FastAPI backend that subscribes to the simulation topics via MQTT and streams real-time state updates to a Vite/React web user interface for visualization.

3. Implementation

3.1. Entities

3.1.1. Base Station

The Base Station operates as the central mission controller and final data sink. Upon receiving a mission start command, it is responsible for broadcasting the mission configuration via the `sim/start` MQTT topic. For structured algorithms like Boustrophedon, it also pre-computes and assigns starting rows to divide the geographical area into manageable bands.

Upon mission completion, it exposes a local endpoint (`sim/base/data_delivery`) where returning drones deliver their aggregated sensor payloads, which are then forwarded to the dashboard for visualization.

3.1.2. Drone

Drones are the autonomous agents executing the coverage mission, governed by a strict internal state machine (IDLE, EXPLORING, COLLECTING, RETURNING, AT_BASE). Navigation is managed by a `Navigator` component that interfaces with a grid representation and dynamically loaded routing strategies. Drones coordinate entirely via decentralized V2X communication, utilizing CAMs for peer awareness and DENMs for grid state updates. To mitigate message loss and ensure eventual map consistency, they also implement a Delay-Tolerant Network (DTN) grid synchronization mechanism that triggers a full map exchange whenever a new peer enters communication range.

3.1.3. Sensor

Sensors are lightweight, passive static nodes distributed across the terrain. During the initial discovery phase, they announce their coordinates once via `sim/announce/{id}` . Afterward, they listen to a dedicated request topic (`sensor/{id}/request`). They only respond with simulated environmental data (such as temperature and humidity) when actively requested by a drone that has navigated into their immediate proximity.

3.2. Proximity Manager

The Proximity Manager serves as the simulation's authoritative physics engine and network controller. It continuously monitors the position of every entity, computes the Euclidean distance between peers from their CAM broadcasts, and uses this information to decide which links are currently valid.

First, it performs global tracking by passively observing the real-time positions of all entities through their CAM messages. Second, it acts as the simulation clock by advancing the system state and publishing connectivity updates on `sim/links` at regular intervals, such as every 500 ms of real time. Third, it enforces the simulated network topology at the data-link layer to model realistic radio propagation and limited communication ranges. To do this, it issues `ebtables` commands directly inside the Docker containers, blocking or unblocking the MAC addresses of specific peers as needed. As a result, drones cannot physically receive MQTT/V2X messages from nodes outside their simulated radio footprint, which makes the decentralized coordination behaviour realistic.

3.3. Cooperative Awareness Messages (CAM)

In Intelligent Transport Systems (ITS), Cooperative Awareness Messages, or CAMs, are used to share real-time information about the presence, status, and position of network stations, whether they are vehicles or infrastructure. In this project, CAM handling is implemented through the open-source `vanetza-nap` middleware, which provides support for the ETSI C-ITS protocol stack.

Rather than working directly with low-level bitstreams, the application layer exchanges human-readable JSON payloads over MQTT. The `vanetza-nap` middleware translates these JSON messages into ETSI-compliant CAMs and also performs the reverse translation when messages are received.

CAMs are transmitted at every simulation tick. Drones publish their messages to the `vanetza/in/cam` topic, while incoming awareness information by neighboring entities is received on `vanetza/out/cam`. Each CAM payload includes the entity's current latitude, longitude, heading, and speed. To distinguish between different kinds of actors in the network, the ETSI `stationType` field is used: drones identify themselves as `stationType = 10` (Special Vehicles), while the base station uses `stationType = 15` (Road Side Unit).

By processing incoming CAMs, each drone maintains a local registry of nearby active peers. This awareness is essential for decentralized coordination, since it supports cooperative coverage and helps reduce the risk of collisions.

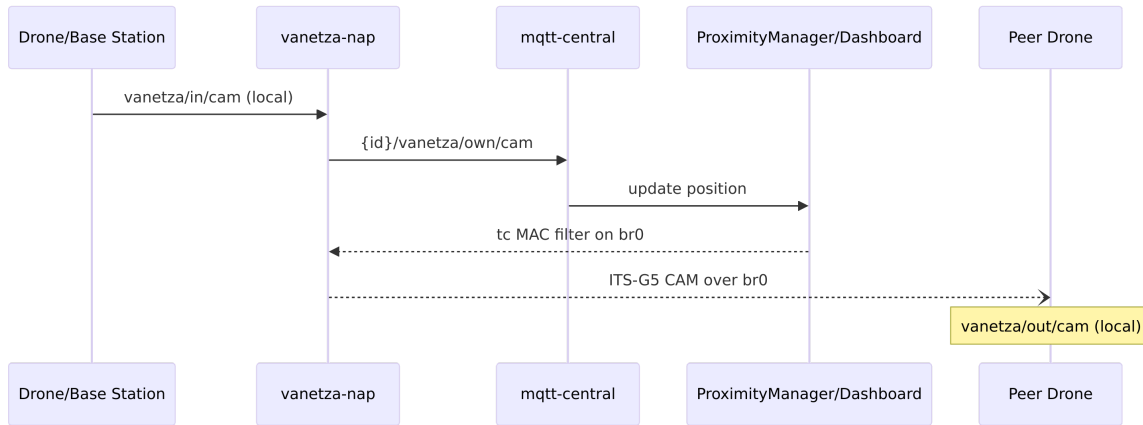


Figure 2: CAM message flow diagram.

3.4. Decentralized Environmental Notification Messages (DENM)

While CAMs provide periodic status updates, DENMs are event-triggered messages that notify other network participants of specific, asynchronous situations, such as hazards or environmental events.

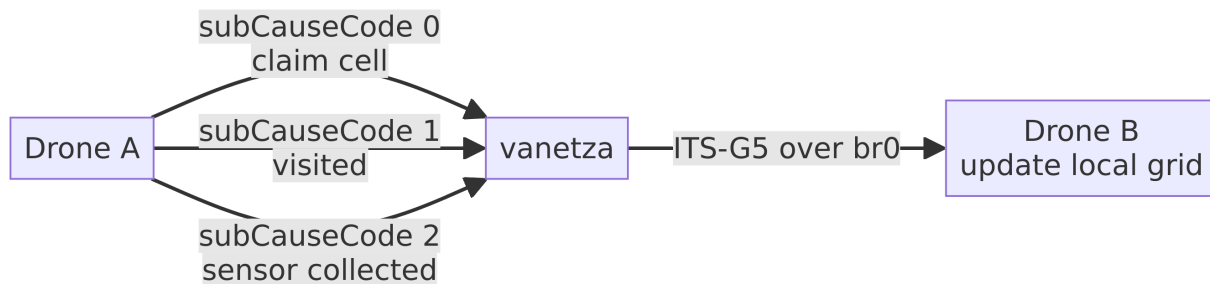


Figure 3: DENM event encoding and dissemination.

In this simulation, DENMs are repurposed to broadcast cell events as drones sweep their assigned areas, using the same `vanetza-nap` JSON-to-ETSI translation pipeline.

The entire geographical area is mapped into a one-dimensional array of grid cells (linear cell indices). When an event occurs within a cell, it is broadcast as a DENM. Because the standard DENM fields are designed for road hazards, this project encodes the spatial event by packing the `cell_index` and a specific `sub_cause` into the `sequenceNumber` field of the DENM payload.

- **Event Sub-Causes:**

- `0 (CLAIMED)` : A drone has selected this cell as its next destination. Broadcasting this prevents other drones from duplicating the effort.

- **1 (VISITED)** : A drone has successfully arrived at the center of the cell and completed its local scan.
- **2 (SENSOR_FOUND)** : A drone has successfully located a ground sensor within the cell and collected its readings.

By receiving DENM updates, drones can update their local map of the environment and avoid revisiting processed cells. Furthermore, to accelerate convergence, when the Proximity Manager detects a new active link between two drones (a first contact), a full grid sync is triggered over the `sim/grid_sync/{id}` MQTT topic.

3.5. Coverage Algorithms

The simulated environment is discretized into a uniform grid. The project implements three distinct, decentralized traversal strategies to achieve total coverage. The user can select the desired strategy when initiating the mission. All algorithms are implemented following a common interface within the drone replicas, ensuring modularity.

3.5.1. Boustrophedon

The Boustrophedon algorithm performs a precomputed lawnmower-style sweep over assigned horizontal bands of the grid.

Each drone calculates a specific horizontal strip of the overall grid to cover. These strips are partitioned dynamically based on the drone's starting position (which is inferred from peer CAM data during the startup phase), so no central planner is required.

Drones sweep their assigned strip row by row, alternating direction at the end of each row. It guarantees complete coverage of the assigned strip with zero overlap.

Pseudo-code:

```
strip <- lawnmower_sequence(start, peers)
loop:
  cell <- first UNKNOWN cell in strip
  if none -> return_to_base()
  move_to(cell)
```

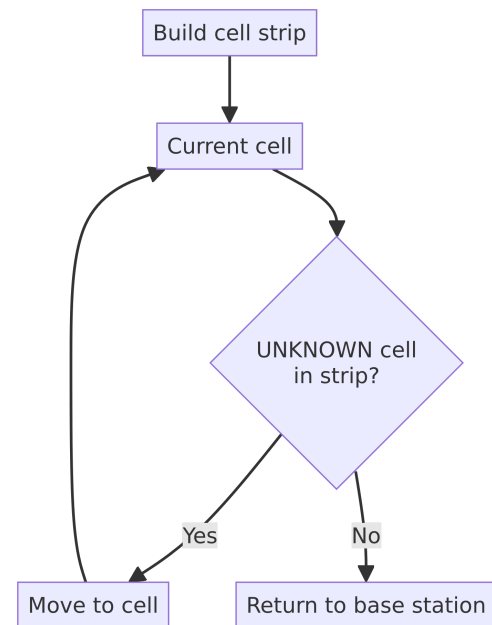


Figure 4.1: Boustrophedon (lawnmower) coverage pattern.

3.5.2. Greedy Nearest

The Greedy Nearest approach is a highly reactive, unconstrained search strategy. There are no predefined zones or assignments, so all drones operate over the entire grid space simultaneously.

At every step, each drone evaluates its local grid map and calculates the Euclidean distance to all remaining unexplored (UNKNOWN) cells, targeting the closest one.

Pseudo-code:

```

loop:
  cell <- nearest UNKNOWN cell in grid
  if none -> return_to_base()
  move_to(cell)

```

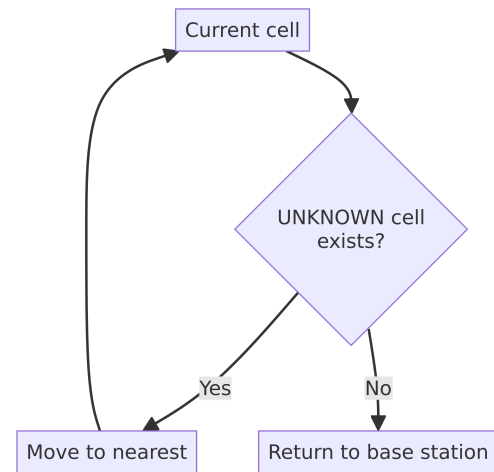


Figure 4.2: Greedy nearest traversal example.

3.5.3. Frontier Exploration

Frontier Exploration is an adaptive strategy that targets the boundary between known (explored) and unknown (unexplored) space. A “frontier cell” is defined as any **UNKNOWN** cell that shares an edge with at least one **VISITED** or **SENSOR_FOUND** cell.

The drone always targets the closest frontier cell. This effectively expands the contiguous explored region outward, reducing wasted travel across already-visited areas. If no frontier exists (at the start of the mission), the algorithm temporarily falls back to the nearest **UNKNOWN** cell until a frontier is established (greedy nearest).

Pseudo-code:

```

loop:
  cell <- nearest frontier cell
        or nearest UNKNOWN cell
  if none -> return_to_base()
  move_to(cell)

```

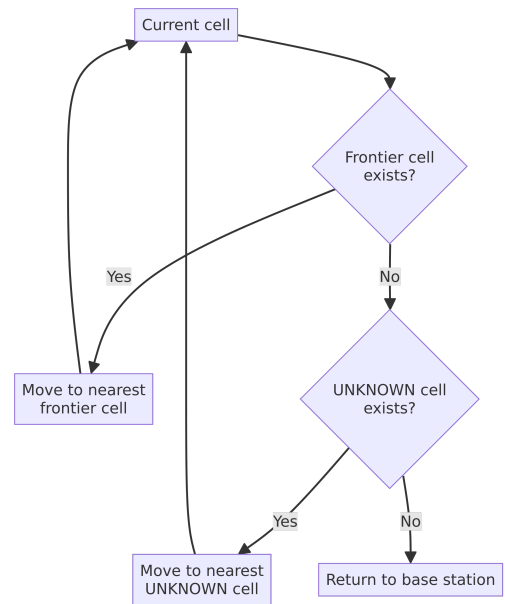


Figure 4.3: Frontier-based exploration example.

3.6. Simulation

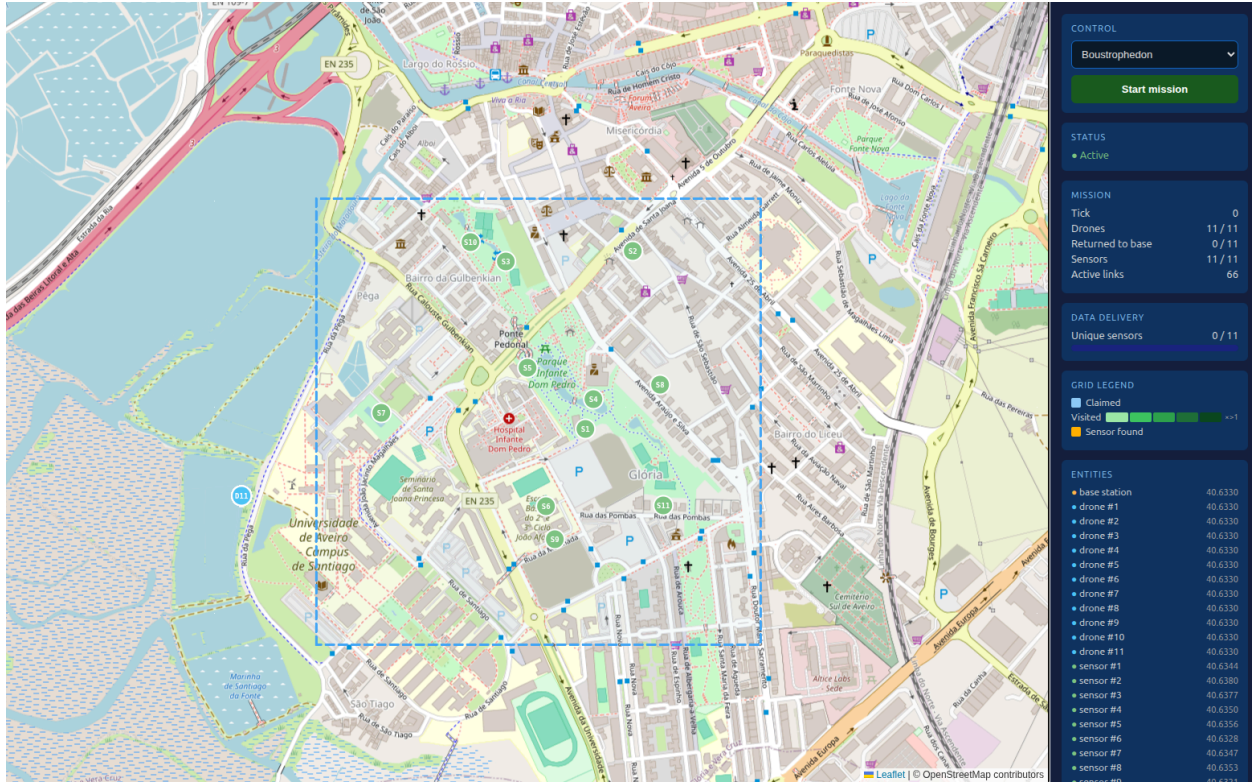


Figure 5: Simulation Dashboard

The lifecycle of a typical simulation run follows a strict sequence of distributed events:

1. **Discovery:** Upon initialization, each entity (drone, sensor, base station) publishes its existence and coordinates to `sim/announce/{id}`.
2. **Mission Start:** The user selects a coverage algorithm via the dashboard and publishes the command to `sim/command/start`. The base station receives this command and broadcasts it to all drones (`sim/start`).
3. **Drone Launch:** Drones initialize their internal grid representations, compute their initial target cells, and begin broadcasting CAM messages periodically.
4. **Grid Sweep & Sync:** Drones navigate the grid. They emit DENM messages to claim cells and mark them as visited. Upon first peer contact (as determined by the Proximity Manager and CAM reception), peers exchange their full grid state via `sim/grid_sync/{id}`.
5. **Data Collection:** When a drone enters the radio range of a sensor, it issues a data request (`sensor/{id}/request`). The sensor responds with its readings, and the drone broadcasts a `SENSOR_FOUND` DENM.
6. **Return & Delivery:** Once a drone's local map indicates that all cells have been covered, it returns to the base station coordinates. Upon arrival, it delivers all aggregated sensor payloads via the `sim/base/data_delivery` topic.

4. Results

To benchmark the algorithms, a standardized simulation scenario was employed:

- **Environment:** 1000×1000m area, discretized into a 20×20 grid (50m cells).
- **Fleet:** 11 drones and 11 static sensors.
- **Radio Limitations:** Drones possess a communication range of 150m.
- **Deterministic parameters:** Random seed set to 42 for reproducible results.

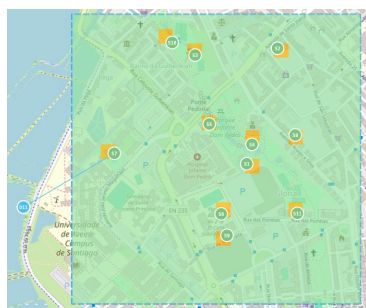


Figure 6.1: Boustrophedon run (coverage result).

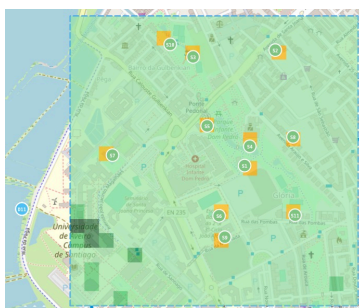


Figure 6.2: Frontier exploration run (coverage result).

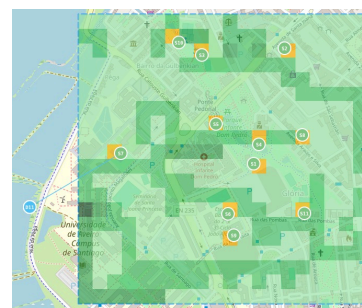


Figure 6.3: Greedy nearest run (coverage result).

Algorithm	Time to Complete (ticks)	Efficiency (Overlap)
Boustrophedon	602	100%
Frontier	751	~90%
Greedy nearest	1018	~68%

Boustrophedon yielded the fastest completion time (602 ticks) with perfect efficiency (zero overlap) by design. However, it is highly rigid, as it requires prior knowledge of all peer positions at the start and possesses no resilience to node failure (if a drone fails, its assigned strip remains uncovered).

Greedy Nearest was the slowest (1018 ticks) and least efficient (~68%). The lack of spatial partitioning causes drones to chase the same targets, leading to redundant coverage. Its performance degrades sharply as the number of drones increases. While simple, this lack of global coordination often leads to criss-crossing paths and significant overlap, making it inefficient when scaled to multiple agents.

In contrast, **Frontier Exploration** emerged as the most balanced and robust strategy (751 ticks, ~90% efficiency). It requires no prior knowledge of the fleet size or deployment positions, limits overlap efficiently, and is inherently fault-tolerant (if a drone fails, its intended area remains as a frontier and will be covered by other active drones).

5. Conclusion

This project demonstrates that ETSI C-ITS messaging standards, specifically CAM and DENM, can be used effectively over MQTT to coordinate a decentralized swarm of autonomous drones in a forest monitoring scenario. By distributing coordination across the fleet rather than relying on a central planner, the system avoids a single point of failure and remains well suited to dynamic and partially connected environments. The simulation also shows that coverage strategy has a strong impact on overall performance: while Boustrophedon achieves the fastest completion time under ideal assumptions, Frontier Exploration offers the most practical balance between efficiency, adaptability, and resilience. In contrast, Greedy Nearest is simple to implement but suffers from redundant movement and lower coverage efficiency.

Taken together, the results indicate that adaptive, locally coordinated behavior is the most promising approach for real-world autonomous data collection missions. Frontier Exploration in particular stands out because it does not require prior knowledge of fleet size or deployment positions, reduces overlap, and can recover more gracefully from drone failure or changing network conditions. This makes it a strong candidate for scenarios where robustness is just as important as speed.

5.1. Future Work

Future work could extend the simulation in several meaningful directions. A particularly relevant improvement would be multi-hop routing, where drones relay DENM messages to neighboring drones and effectively form a mesh network. This would allow information about claimed, visited, or sensor-containing cells to propagate beyond direct communication range, giving each drone a much more up-to-date local grid and improving coordination in larger or more fragmented deployments.

Energy-aware scheduling could also be introduced by modeling battery consumption and forcing drones to return to the base station for recharging before resuming coverage.

In addition, obstacle avoidance would make the environment more realistic by adding blocked terrain and requiring path planning around non-traversable cells.

Finally, a more sophisticated radio model based on path loss and shadowing would better capture communication behavior in forested environments and strengthen the realism of the overall system.